SQLAlchemy 2.0 Cheat Sheet

A practical quick reference for beginners & intermediate users
Ordered from the most commonly used functions to the least

EVERY EXAMPLE IN THIS GUIDE USES THIS SETUP

```python
from sqlalchemy import create_engine, select, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

engine = create_engine("postgresql+psycopg2://user:pw@localhost/hr")

class Base(DeclarativeBase): pass

class Employee(Base):
    __tablename__ = "employees"
    id:     Mapped[str] = mapped_column("employee_id", primary_key=True)
    name:   Mapped[str] = mapped_column("full_name")
    salary: Mapped[float | None]
    ...                     # 14 columns, 247 rows
```

FOUR IDEAS THAT EXPLAIN MOST OF MATPLOTLIB

`Engine`	One per app. Holds the connection pool - create it once.
`Session`	Your workspace for ORM objects. Open, use, close.
`Core vs ORM`	Core builds SQL; the ORM maps rows to Python objects.
`select()`	The 2.0 way. Works for both Core and ORM queries.

HOW THE SAMPLE OUTPUTS WERE PRODUCED

PostgreSQL 17.10	live server in the sandbox	97 / 97 ran
SQLite 3.46	same Python code, different URL	97 / 97 ran
Identical output	92 of 97 snippets, byte for byte	that is the point of an ORM

Blue bar = code you type **Green bar** = real value returned **97** examples · **13** topics

Companion to the Pandas, NumPy & Scikit-Learn cheat sheets — all built on the same HR dataset.

Table of Contents

1. Connecting to a Database

9 functions

Create the engine that talks to your database.

1.1

```
# Checks the version you have
import sqlalchemy
sqlalchemy.__version__
```

SAMPLE OUTPUT

```
2.0.51
```

1.2

```
engine = create_engine("sqlite:///hr.db")    # Connects to a SQLite file
```

SAMPLE OUTPUT

```
sqlite
```

1.3

```
# Connects to PostgreSQL
engine = create_engine(
    "postgresql+psycopg2://user:pw@localhost:5432/hr")
```

SAMPLE OUTPUT

```
postgresql+psycopg2 -> PostgreSQL
```

Watch out: Format is dialect+driver://user:password@host:port/database

1.4

```
# Connects to MySQL
engine = create_engine(
    "mysql+pymysql://user:pw@localhost:3306/hr")
```

SAMPLE OUTPUT

```
mysql+pymysql -> MySQL / MariaDB
```

1.5

```
# Connects to SQL Server
engine = create_engine(
    "mssql+pyodbc://user:pw@server/hr?driver=ODBC+Driver+17+for+SQL+Server")
```

SAMPLE OUTPUT

```
mssql+pyodbc -> Microsoft SQL Server
```

Watch out: Same Python code afterwards - only this URL changes per engine.

1.6

```
# Opens and closes a connection safely
with engine.connect() as conn:
    result = conn.execute(text("SELECT 1"))
    value = result.scalar()
```

SAMPLE OUTPUT

```
1
```

1.7

```
engine.dialect.name    # Shows which database you are on
```

SAMPLE OUTPUT

```
postgresql
```

1.8

```
create_engine(DB_URL, echo=True)    # Prints every SQL statement it runs
```

SAMPLE OUTPUT

```
echo=True logs SQL to the console
```

1.9

```
create_engine(DB_URL, pool_size=10, max_overflow=20)    # Controls how many connections to keep
```

SAMPLE OUTPUT

```
pool_size=10 -> 10 reusable connections
```

2. Defining Tables & Models

11 functions

Describe your tables as Python classes.

2.1

```python
# The parent class for all models
class Base(DeclarativeBase):
    pass
```

SAMPLE OUTPUT

```
every model inherits from Base
```

2.2

```python
# Maps a class to a table
class Employee(Base):
    __tablename__ = "employees"
    id: Mapped[str] = mapped_column(String(10), primary_key=True)
    name: Mapped[str] = mapped_column(String(60))
```

SAMPLE OUTPUT

```
('employees', ['id', 'name'])
```

2.3

```python
Employee.__tablename__    # The table name behind a model
```

SAMPLE OUTPUT

```
employees
```

2.4

```python
Employee.__table__.columns.keys()    # Lists the mapped column names
```

SAMPLE OUTPUT

```
['employee_id', 'full_name', 'gender', 'department', 'position', 'hired_date',
'employment_type', 'office_location', 'salary', 'age', 'performance_rating']
```

2.5

```python
Base.metadata.create_all(engine)    # Creates every table that is missing
```

SAMPLE OUTPUT

```
CREATE TABLE runs only for missing tables
```

2.6

```python
Base.metadata.tables.keys()    # All tables SQLAlchemy knows about
```

SAMPLE OUTPUT

```
dict_keys(['departments', 'employees'])
```

2.7

```python
# Core style: a table without a class
employees = Table("employees", MetaData(),
                  Column("employee_id", String(10), primary_key=True),
                  Column("full_name", String(60)))
```

SAMPLE OUTPUT

```
['employee_id', 'full_name']
```

2.8

```python
mapped_column(String(60), nullable=False, index=True)    # Adds rules to a column
```

SAMPLE OUTPUT

```
NOT NULL plus an index
```

2.9

```python
mapped_column(ForeignKey("departments.name"))    # Links a column to another table
```

SAMPLE OUTPUT

```
creates the FOREIGN KEY constraint
```

2.10

```python
inspect(engine).get_table_names()    # Lists real tables in the database
```

SAMPLE OUTPUT

```
['departments', 'employees']
```

2.11

```python
# Reads the real columns from the database
[c["name"] for c in inspect(engine).get_columns("employees")]
```

SAMPLE OUTPUT

```
['employee_id', 'full_name', 'gender', 'department', 'position', 'hired_date',
'employment_type', 'office_location', 'salary', 'age', 'email', 'phone_number',
'performance_rating', 'manager']
```

3. Querying with SELECT

8 functions

Read rows back out again.

3.1

```
# Gets every row as objects
with Session(engine) as s:
    rows = s.scalars(select(Employee)).all()
len(rows)
```

SAMPLE OUTPUT

```
247
```

3.2

```
# Gets the first matching object
with Session(engine) as s:
    e = s.scalars(select(Employee).limit(1)).first()
(e.id, e.name, e.dept_name)
```

SAMPLE OUTPUT

```
('PH-1466', 'Alvin Aglipay', 'Engineering')
```

3.3

```
# Gets only the columns you need
with Session(engine) as s:
    rows = s.execute(
        select(Employee.name, Employee.salary).limit(4)).all()
rows
```

SAMPLE OUTPUT

```
('Alvin Aglipay', Decimal('69500.00'))
('Alvin Alcantara', Decimal('91500.00'))
('Alvin Luna', Decimal('100000.00'))
('Alvin Ramos', Decimal('102000.00'))
```

3.4

```
# Looks a row up by primary key
with Session(engine) as s:
    e = s.get(Employee, "PH-1466")
(e.name, e.position)
```

SAMPLE OUTPUT

```
('Alvin Aglipay', 'Data Engineer')
```

Watch out: s.get() checks the identity map first, so it can skip the database.

3.5

```
# Counts the rows
with Session(engine) as s:
    n = s.scalar(select(func.count()).select_from(Employee))
n
```

SAMPLE OUTPUT

```
247
```

3.6

```
# Lists unique values only
with Session(engine) as s:
    vals = s.scalars(
        select(Employee.dept_name).distinct()).all()
sorted(vals)
```

SAMPLE OUTPUT

```
['Engineering', 'Finance', 'Human Resources', 'IT', 'Marketing', 'Operations', 'Sales']
```

3.7

```
# Skips rows then takes some
with Session(engine) as s:
    rows = s.scalars(
        select(Employee).limit(3).offset(2)).all()
[e.name for e in rows]
```

SAMPLE OUTPUT

```
['Alvin Luna', 'Alvin Ramos', 'Amalia Alonzo']
```

Watch out: SQLAlchemy writes LIMIT/OFFSET or TOP/FETCH to match your database.

3.8

```
print(select(Employee.name).where(Employee.age > 40))    # Shows the SQL it will send
```

SAMPLE OUTPUT

```
SELECT employees.full_name
FROM employees
WHERE employees.age > :age_1
```

4. Filtering with WHERE

10 functions

Keep only the rows you want.

4.1

```
# Matches one exact value
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.dept_name == "Engineering")).all()
len(rows)
```

SAMPLE OUTPUT

```
58
```

Watch out: Use == not =, and Python "is None" becomes .is_(None).

4.2

```
# Compares a number
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.salary > 100000)).all()
len(rows)
```

SAMPLE OUTPUT

```
55
```

4.3

```
# Several conditions, all must match
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.salary > 80000, Employee.age < 30)).all()
[(e.name, e.age) for e in rows][:3]
```

SAMPLE OUTPUT

```
[('Amalia Alonzo', 24), ('Fernando Ilagan', 22), ('Hector Zamora', 25)]
```

Watch out: Commas mean AND. For OR you must use or_().

4.4

```
# Either condition can match
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(or_(Employee.age < 25, Employee.age > 58))).all()
len(rows)
```

SAMPLE OUTPUT

```
20
```

4.5

```
# Matches any value in a list
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.office.in_(["Cebu City", "Clark"]))).all()
len(rows)
```

SAMPLE OUTPUT

```
64
```

4.6

```
# Matches a range, ends included
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.age.between(30, 35))).all()
len(rows)
```

SAMPLE OUTPUT

```
37
```

4.7

```
# Finds text containing a word
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.position.like("%Manager%"))).all()
len(rows)
```

SAMPLE OUTPUT

```
49
```

4.8

```
# Finds missing values
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.salary.is_(None))).all()
len(rows)
```

SAMPLE OUTPUT

```
12
```

Watch out: .is_(None) becomes IS NULL. Writing == None works but linters hate it.

4.9

```
# Excludes a value
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.dept_name != "Sales")).all()
len(rows)
```

SAMPLE OUTPUT

```
216
```

4.10

```
# Ignores upper and lower case
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.name.ilike("alvin%"))).all()
[e.name for e in rows][:3]
```

SAMPLE OUTPUT

```
['Alvin Aglipay', 'Alvin Alcantara', 'Alvin Luna']
```

Watch out: ilike() works everywhere: on MySQL/SQL Server it becomes LOWER(x) LIKE.

5. Sorting & Limiting

5 functions

Put the rows in order.

5.1

```
# Highest values first
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .order_by(Employee.salary.desc().nullslast())
        .limit(3)).all()
[(e.name, float(e.salary)) for e in rows]
```

SAMPLE OUTPUT

```
[('Wilfredo Mendoza', 184000.0), ('Fernando Castillo', 160000.0), ('Teresita Espiritu',
159000.0)]
```

Watch out: Postgres puts NULLs FIRST on DESC, so add .nullslast() to get real values.

5.2

```
# Lowest values first
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.salary.isnot(None))
        .order_by(Employee.salary).limit(3)).all()
[(e.name, float(e.salary)) for e in rows]
```

SAMPLE OUTPUT

```
[('Bea Cruz', 33000.0), ('Divina Espiritu', 34000.0), ('Bea Quijano', 34000.0)]
```

5.3

```
# Sorts by two columns
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .order_by(Employee.dept_name, Employee.name).limit(3)).all()
[(e.dept_name, e.name) for e in rows]
```

SAMPLE OUTPUT

```
[('Engineering', 'Alvin Aglipay'), ('Engineering', 'Alvin Alcantara'), ('Engineering',
'Alvin Luna')]
```

5.4

```
# Pushes empty values to the end
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .order_by(Employee.salary.desc().nullslast())
        .limit(3)).all()
[(e.name, float(e.salary)) for e in rows]
```

SAMPLE OUTPUT

```
[('Wilfredo Mendoza', 184000.0), ('Fernando Castillo', 160000.0), ('Teresita Espiritu',
159000.0)]
```

Watch out: nullslast() is emulated on MySQL, which has no NULLS LAST of its own.

5.5

```
# The desc() function form
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .order_by(desc(Employee.age)).limit(3)).all()
[(e.name, e.age) for e in rows]
```

SAMPLE OUTPUT

```
[('Amalia Fernandez', 59), ('Angelica Aglipay', 59), ('Carlo Bautista', 59)]
```

6. Aggregation & GROUP BY

8 functions

Summarise many rows into one.

6.1

```python
# Calculates the average
with Session(engine) as s:
    avg = s.scalar(select(func.avg(Employee.salary)))
round(float(avg), 2)
```

SAMPLE OUTPUT

```
83014.89
```

6.2

```python
# Smallest and largest values
with Session(engine) as s:
    rows = s.execute(
        select(func.min(Employee.salary),
               func.max(Employee.salary))).one()
[float(v) for v in rows]
```

SAMPLE OUTPUT

```
[33000.0, 184000.0]
```

6.3

```python
# Counts rows in each group
with Session(engine) as s:
    rows = s.execute(
        select(Employee.dept_name, func.count().label("staff"))
        .group_by(Employee.dept_name)
        .order_by(desc("staff"), Employee.dept_name)).all()
rows
```

SAMPLE OUTPUT

```
('Engineering', 58)
('IT', 40)
('Finance', 38)
('Operations', 31)
('Sales', 31)
('Human Resources', 26)
('Marketing', 23)
```

6.4

```python
# Average for each group
with Session(engine) as s:
    rows = s.execute(
        select(Employee.dept_name,
               func.round(cast(func.avg(Employee.salary), Numeric), 2))
        .group_by(Employee.dept_name)
        .order_by(Employee.dept_name)).all()
rows[:4]
```

SAMPLE OUTPUT

```
('Engineering', Decimal('98419.64'))
('Finance', Decimal('88986.11'))
('Human Resources', Decimal('76980.77'))
('IT', Decimal('79657.89'))
```

6.5

```python
# Filters the groups, not the rows
with Session(engine) as s:
    rows = s.execute(
        select(Employee.dept_name, func.count().label("n"))
        .group_by(Employee.dept_name)
        .having(func.count() > 30)
        .order_by(Employee.dept_name)).all()
rows
```

SAMPLE OUTPUT

```
('Engineering', 58)
('Finance', 38)
('IT', 40)
('Operations', 31)
('Sales', 31)
```

6.6

```python
# Counts unique values
with Session(engine) as s:
    n = s.scalar(
        select(func.count(func.distinct(Employee.dept_name))))
n
```

SAMPLE OUTPUT

```
7
```

6.7

```python
# Groups by two columns
with Session(engine) as s:
    rows = s.execute(
        select(Employee.dept_name, Employee.gender,
               func.count().label("n"))
        .group_by(Employee.dept_name, Employee.gender)
        .order_by(Employee.dept_name)).all()
rows[:5]
```

SAMPLE OUTPUT

```
('Engineering', 'Female', 33)
('Engineering', 'Male', 25)
('Finance', 'Female', 20)
('Finance', 'Male', 18)
('Human Resources', 'Female', 13)
```

6.8

```python
# Counts sideways with CASE
with Session(engine) as s:
    rows = s.execute(
        select(Employee.dept_name,
               func.sum(case((Employee.gender == "Female", 1),
                             else_=0)).label("women"))
        .group_by(Employee.dept_name)
        .order_by(Employee.dept_name)).all()
rows[:4]
```

SAMPLE OUTPUT

```
('Engineering', 33)
('Finance', 20)
('Human Resources', 13)
('IT', 20)
```

7. Joins & Relationships

7 functions

Pull data from two tables at once.

7.1

```
# Joins using the mapped relationship
with Session(engine) as s:
    rows = s.execute(
        select(Employee.name, Department.head)
        .join(Department).limit(3)).all()
rows
```

SAMPLE OUTPUT

```
('Alvin Aglipay', 'Ramon Villanueva')
('Alvin Alcantara', 'Ramon Villanueva')
('Alvin Luna', 'Ramon Villanueva')
```

Watch out: With a ForeignKey defined, SQLAlchemy works out the ON clause itself.

7.2

```
# Joins with an explicit condition
with Session(engine) as s:
    rows = s.execute(
        select(Employee.name, Department.floor)
        .join(Department, Employee.dept_name == Department.name)
        .limit(3)).all()
rows
```

SAMPLE OUTPUT

```
('Alvin Aglipay', 5)
('Alvin Alcantara', 5)
('Alvin Luna', 5)
```

7.3

```
# Keeps groups that have no matches
with Session(engine) as s:
    rows = s.execute(
        select(Department.name, func.count(Employee.id))
        .outerjoin(Employee)
        .group_by(Department.name)
        .order_by(func.count(Employee.id), Department.name)).all()
rows[:4]
```

SAMPLE OUTPUT

```
('Logistics', 0)
('Marketing', 23)
('Human Resources', 26)
('Operations', 31)
```

7.4

```
# Follows a relationship like an attribute
with Session(engine) as s:
    e = s.get(Employee, "PH-1466")
    head = e.dept.head
(e.name, e.dept_name, head)
```

SAMPLE OUTPUT

```
('Alvin Aglipay', 'Engineering', 'Ramon Villanueva')
```

Watch out: This fires a second SELECT the moment you touch e.dept (lazy loading).

7.5

```
# Reads the other side of the link
with Session(engine) as s:
    d = s.get(Department, "Engineering")
    n = len(d.employees)
n
```

SAMPLE OUTPUT

```
58
```

7.6

```
# Loads related rows in one go
with Session(engine) as s:
    rows = s.scalars(select(Department)
        .options(selectinload(Department.employees))).all()
sum(len(d.employees) for d in rows)
```

SAMPLE OUTPUT

```
247
```

Watch out: THE fix for the N+1 problem: one extra query instead of one per row.

7.7

```python
# Joins a table to itself
mgr = aliased(Employee)
with Session(engine) as s:
    rows = s.execute(
        select(Employee.name, mgr.name)
        .join(mgr, Employee.dept_name == mgr.dept_name)
        .where(mgr.position.like("%Manager%"))
        .order_by(Employee.name, mgr.name).limit(3)).all()
rows
```

SAMPLE OUTPUT

```
('Alvin Aglipay', 'Dennis Mendoza')
('Alvin Aglipay', 'Divina Soriano')
('Alvin Aglipay', 'Kevin Valdez')
```

8. Inserting Data

5 functions

Put new rows into the table.

8.1

```
# Adds one new object
with Session(engine) as s:
    s.add(Department(name="Legal", head="Nora Aunor", floor=8))
    s.commit()
```

SAMPLE OUTPUT

```
1 row committed
```

8.2

```
# Adds several objects at once
with Session(engine) as s:
    s.add_all([Department(name="Design", head="Ella Cruz", floor=9),
               Department(name="Support", head="Ben Chan", floor=10)])
    s.commit()
    total = s.scalar(select(func.count()).select_from(Department))
total
```

SAMPLE OUTPUT

```
10
```

8.3

```
# Sends the INSERT without committing
with Session(engine) as s:
    d = Department(name="Legal", head="Nora Aunor", floor=8)
    s.add(d)
    s.flush()
    name_now = d.name
name_now
```

SAMPLE OUTPUT

```
Legal
```

Watch out: flush() writes to the database; commit() makes it permanent.

8.4

```
# Fast bulk insert, Core style
with Session(engine) as s:
    s.execute(insert(Department),
              [{"name": "Legal", "head": "N. Aunor", "floor": 8},
               {"name": "Design", "head": "E. Cruz", "floor": 9}])
    s.commit()
    total = s.scalar(select(func.count()).select_from(Department))
total
```

SAMPLE OUTPUT

```
10
```

8.5

```
# Reloads values the database set
with Session(engine) as s:
    d = Department(name="Legal", head="Nora Aunor", floor=8)
    s.add(d)
    s.commit()
    s.refresh(d)
    saved = d.floor
saved
```

SAMPLE OUTPUT

```
8
```

9. Updating & Deleting

6 functions

Change or remove existing rows.

9.1

```
# Edits an object then commits
with Session(engine) as s:
    d = s.get(Department, "Finance")
    d.floor = 12
    s.commit()
    now = s.get(Department, "Finance").floor
now
```

SAMPLE OUTPUT

```
12
```

Watch out: The ORM notices the change and writes the UPDATE for you.

9.2

```
# Updates many rows in one statement
with Session(engine) as s:
    s.execute(update(Department)
        .where(Department.name == "Finance")
        .values(floor=15))
    s.commit()
    now = s.get(Department, "Finance").floor
now
```

SAMPLE OUTPUT

```
15
```

9.3

```
# Counts how many rows changed
with Session(engine) as s:
    n = s.execute(update(Department)
        .where(Department.floor > 4)
        .values(floor=99)).rowcount
    s.commit()
n
```

SAMPLE OUTPUT

```
4
```

9.4

```
# Deletes one object
with Session(engine) as s:
    d = s.get(Department, "Logistics")
    s.delete(d)
    s.commit()
    left = s.scalar(select(func.count()).select_from(Department))
left
```

SAMPLE OUTPUT

```
7
```

9.5

```
# Deletes many rows at once
with Session(engine) as s:
    n = s.execute(delete(Department)
        .where(Department.name == "Logistics")).rowcount
    s.commit()
n
```

SAMPLE OUTPUT

```
1
```

Watch out: A row still referenced by a ForeignKey cannot be deleted - clear the children first.

9.6

```
# Undoes changes before committing
with Session(engine) as s:
    d = s.get(Department, "Finance")
    d.floor = 99
    s.rollback()
    back = s.get(Department, "Finance").floor
back
```

SAMPLE OUTPUT

```
3
```

Watch out: Every Session runs in a transaction - rollback() throws the changes away.

10. Sessions & Transactions

6 functions

Control when changes become permanent.

10.1

```
# The standard way to open a session
with Session(engine) as s:
    n = s.scalar(select(func.count()).select_from(Employee))
n
```

SAMPLE OUTPUT

```
247
```

Watch out: The with block always closes the session, even if an error happens.

10.2

```
# Commits automatically at the end
with Session(engine) as s, s.begin():
    s.add(Department(name="Legal", head="N. Aunor", floor=8))
# commits automatically on a clean exit
```

SAMPLE OUTPUT

```
committed on exit, rolled back on error
```

10.3

```
# Undoes everything if it fails
with Session(engine) as s:
    try:
        s.add(Department(name="Finance", head="dup", floor=1))
        s.commit()
    except Exception:
        s.rollback()
        outcome = "rolled back"
outcome
```

SAMPLE OUTPUT

```
rolled back
```

10.4

```
# Core style with automatic commit
with engine.begin() as conn:
    n = conn.execute(text("SELECT COUNT(*) FROM employees")).scalar()
n
```

SAMPLE OUTPUT

```
247
```

10.5

```
# Shows objects with unsaved changes
with Session(engine) as s:
    e = s.get(Employee, "PH-1466")
    e.age = 99
    dirty = bool(s.dirty)
    s.rollback()
dirty
```

SAMPLE OUTPUT

```
True
```

10.6

```
# Removes an object from the session
with Session(engine) as s:
    e = s.get(Employee, "PH-1466")
    s.expunge(e)
    detached = e not in s
detached
```

SAMPLE OUTPUT

```
True
```

11. Raw SQL & Pandas

6 functions

Drop to plain SQL when you need to.

11.1

```
# Runs a plain SQL string
with engine.connect() as conn:
    rows = conn.execute(text(
        "SELECT department, COUNT(*) FROM employees "
        "GROUP BY department ORDER BY department")).all()
rows[:4]
```

SAMPLE OUTPUT

```
('Engineering', 58)
('Finance', 38)
('Human Resources', 26)
('IT', 40)
```

11.2

```
# Passes values safely as parameters
with engine.connect() as conn:
    rows = conn.execute(
        text("SELECT full_name FROM employees "
             "WHERE department = :d LIMIT 3"),
        {"d": "Engineering"}).scalars().all()
rows
```

SAMPLE OUTPUT

```
['Alvin Aglipay', 'Alvin Alcantara', 'Alvin Luna']
```

Watch out: Always use :name parameters. Never build SQL with f-strings.

11.3

```
# Reads a query straight into pandas
import pandas as pd
df = pd.read_sql(select(Employee.name, Employee.salary).limit(4),
                 engine)
df
```

SAMPLE OUTPUT

```
         full_name    salary
0    Alvin Aglipay   69500.0
1  Alvin Alcantara   91500.0
2       Alvin Luna  100000.0
3      Alvin Ramos  102000.0
```

11.4

```
# Reads raw SQL into a DataFrame
import pandas as pd
df = pd.read_sql_query(
    text("SELECT department, COUNT(*) AS n FROM employees "
         "GROUP BY department ORDER BY department"), engine)
df.head(4)
```

SAMPLE OUTPUT

```
        department   n
0      Engineering  58
1          Finance  38
2  Human Resources  26
3               IT  40
```

11.5

```
# Writes a DataFrame into a table
import pandas as pd
small = pd.DataFrame({"name": ["Legal"], "head": ["N. Aunor"],
                      "floor": [8]})
small.to_sql("departments", engine, if_exists="append", index=False)
```

SAMPLE OUTPUT

```
1
```

11.6

```
# Turns a result row into a dictionary
with engine.connect() as conn:
    row = conn.execute(text("SELECT * FROM employees LIMIT 1")).first()
    as_dict = dict(row._mapping)
list(as_dict)[:6]
```

SAMPLE OUTPUT

```
['employee_id', 'full_name', 'gender', 'department', 'position', 'hired_date']
```

12. Inspecting & Debugging

8 functions

See what SQLAlchemy is actually doing.

12.1

```
print(select(Employee).where(Employee.age > 40))    # Prints the SQL for a query
```

SAMPLE OUTPUT

```
SELECT employees.employee_id, employees.full_name, employees.gender, employees.department,
employees.position, employees.hired_date, employees.employment_type,
employees.office_location, employees.salary, employees.age, employees.performance_rating
FROM employees
WHERE employees.age > :age_1
```

12.2

```
# Shows the SQL with real values
stmt = select(Employee.name).where(Employee.salary > 100000)
print(stmt.compile(engine,
      compile_kwargs={"literal_binds": True}))
```

SAMPLE OUTPUT

```
SELECT employees.full_name
FROM employees
WHERE employees.salary > 100000
```

Watch out: Handy for pasting into a database client to test by hand.

12.3

```
inspect(engine).get_table_names()    # Lists the tables that exist
```

SAMPLE OUTPUT

```
['departments', 'employees']
```

12.4

```
inspect(engine).get_pk_constraint("employees")    # Shows the primary key
```

SAMPLE OUTPUT

```
{'constrained_columns': ['employee_id'], 'name': 'employees_pkey', 'comment': None,
'dialect_options': {'postgresql_include': []}}
```

12.5

```
# Shows which tables it links to
[fk["referred_table"] for fk in
 inspect(engine).get_foreign_keys("employees")]
```

SAMPLE OUTPUT

```
['departments']
```

12.6

```
inspect(Employee).mapper.column_attrs.keys()    # Lists the attributes on a model
```

SAMPLE OUTPUT

```
['id', 'name', 'gender', 'dept_name', 'position', 'hired', 'emp_type', 'office', 'salary',
'age', 'rating']
```

12.7

```
# Checks if an object is saved
with Session(engine) as s:
    e = s.get(Employee, "PH-1466")
    state = inspect(e).persistent
state
```

SAMPLE OUTPUT

```
True
```

12.8

```
engine.url.render_as_string(hide_password=True)    # Shows the connection URL safely
```

SAMPLE OUTPUT

```
postgresql+psycopg2://hr@127.0.0.1:55432/hr
```

Watch out: render_as_string hides the password; str(url) would leak it into logs.

13. Advanced Patterns

8 functions

Techniques worth knowing as you grow.

13.1

```
# Uses a query inside another query
with Session(engine) as s:
    sub = (select(Employee.dept_name,
                  func.avg(Employee.salary).label("avg_pay"))
           .group_by(Employee.dept_name).subquery())
    rows = s.execute(select(sub.c.dept_name, sub.c.avg_pay)
                     .order_by(sub.c.avg_pay.desc()).limit(3)).all()
[(r[0], round(float(r[1]))) for r in rows]
```

SAMPLE OUTPUT

```
[('Engineering', 98420), ('Finance', 88986), ('IT', 79658)]
```

13.2

```
# Names a query block with a CTE
with Session(engine) as s:
    cte = (select(Employee.dept_name,
                  func.count().label("n"))
           .group_by(Employee.dept_name).cte("counts"))
    rows = s.execute(select(cte.c.dept_name, cte.c.n)
                     .where(cte.c.n > 35)).all()
rows
```

SAMPLE OUTPUT

```
('Engineering', 58)
('Finance', 38)
('IT', 40)
```

13.3

```
# Ranks rows with a window function
row_no = func.row_number().over(
    partition_by=Employee.dept_name,
    order_by=Employee.salary.desc().nullslast())
with Session(engine) as s:
    rows = s.execute(
        select(Employee.dept_name, Employee.name, row_no.label("rn"))
        .order_by(Employee.dept_name, row_no).limit(4)).all()
rows
```

SAMPLE OUTPUT

```
('Engineering', 'Wilfredo Mendoza', 1)
('Engineering', 'Teresita Espiritu', 2)
('Engineering', 'Divina Soriano', 3)
('Engineering', 'Rosario Bonifacio', 4)
```

13.4

```
# Adds if-then logic to a query
with Session(engine) as s:
    rows = s.execute(
        select(Employee.name,
               case((Employee.salary > 100000, "High"),
                    (Employee.salary > 70000, "Medium"),
                    else_="Low").label("band")).limit(4)).all()
rows
```

SAMPLE OUTPUT

```
('Alvin Aglipay', 'Low')
('Alvin Alcantara', 'Medium')
('Alvin Luna', 'Medium')
('Alvin Ramos', 'High')
```

13.5

```
# Filters on a related table
with Session(engine) as s:
    rows = s.scalars(select(Employee)
        .where(Employee.dept.has(Department.floor > 4))
        .limit(3)).all()
[e.name for e in rows]
```

SAMPLE OUTPUT

```
['Alvin Aglipay', 'Alvin Alcantara', 'Alvin Luna']
```

13.6

```
# Finds parents by their children
with Session(engine) as s:
    rows = s.scalars(select(Department)
        .where(Department.employees.any(Employee.salary > 150000))
        ).all()
[d.name for d in rows]
```

SAMPLE OUTPUT

```
['Engineering', 'IT']
```

13.7

```
# Uses a subquery inside IN
stmt = select(Employee).where(
    Employee.dept_name.in_(select(Department.name)
                           .where(Department.floor > 3)))
with Session(engine) as s:
    n = len(s.scalars(stmt).all())
n
```

SAMPLE OUTPUT

```
152
```

13.8

```
Index("idx_dept", Employee.dept_name)    # Defines an index on a column
```

SAMPLE OUTPUT

```
Index(idx_dept) -> created by create_all()
```

14. Portability Reference

16 functions

What SQLAlchemy hides, and what it cannot.

SQLAlchemy writes the right SQL for you

Topic	What you write once	What actually happens
Paging	`.limit(5).offset(10)`	LIMIT/OFFSET on MySQL & Postgres, OFFSET...FETCH on SQL Server
Case-insensitive	`.ilike('alvin%')`	ILIKE on Postgres, LOWER(x) LIKE LOWER(y) elsewhere
NULLs last	`.desc().nullslast()`	NULLS LAST on Postgres, an extra IS NULL sort key on MySQL
Quoting names	`Column("order")`	`order` on MySQL, "order" on Postgres, [order] on SQL Server
String join	`col_a + col_b (strings)`	CONCAT() on MySQL, || on Postgres, + on SQL Server
Auto-increment	`mapped_column(primary_key=True)`	AUTO_INCREMENT / SERIAL / IDENTITY(1,1) as appropriate
Boolean values	`Mapped[bool]`	TINYINT(1) on MySQL, BOOLEAN on Postgres, BIT on SQL Server
Placeholders	`text('... :name'), {'name': v}`	%s, $1 or ? depending on the driver - always escaped safely

Things you still have to think about

Topic	What you write once	What actually happens
Numeric type	`func.avg(col) returns Decimal`	Postgres gives Decimal, SQLite gives float - cast if you need one
round(x, 2)	`func.round(cast(x, Numeric), 2)`	Postgres has no round(double precision, int) - cast first
NULL sort order	`add .nullslast() explicitly`	Postgres sorts NULLs first on DESC; MySQL sorts them last
Row order	`always add .order_by()`	Without it every engine may return rows in a different order
Constraint names	`inspect(...).get_pk_constraint()`	Postgres reports 'employees_pkey'; SQLite reports None
Date functions	`func.extract('year', col)`	Portable, but exotic date maths still needs dialect-specific SQL
Upsert	`dialect-specific insert()`	from sqlalchemy.dialects.postgresql import insert - not portable
Engine name	`engine.dialect.name`	Branch on this when you genuinely need engine-specific behaviour